

Portál pre referenda Slovenských krajov

Zámer

Portál pre referenda slovenských krajov umožňuje používateľom prehľadne sledovať aktuálne a nadchádzajúce referendá v ich kraji. Okrem toho, že môžu vidieť výsledky doterajších hlasovaní, majú aj možnosť sa do nich zapojiť po prihlásení. Referendá predstavujú otázky, na ktoré sa odpovedá jednoducho "áno" alebo "nie", pričom niektoré referendá môžu mať aj viacero otázok.

Administrátori majú v systéme širšie právomoci, vrátane možnosti vytvárať nové referendá, posilať notifikácie a zverejňovať ich výsledky. Systém navyše umožňuje preferovať účasť domácich obyvateľov v danom kraji, čím sa podporuje ich aktívna účasť v demokratickom procese.

Existujú tri typy referend -Jednoduché, kde stačí získať viacero hlasov za "áno" ako za "nie". Potom je tu kvôťované referendum, kde sú stanovené kvóty pre minimálny počet účastníkov a percentuálne zastúpenie "áno". Posledným typom je Podmienené referendum, kde okrem kvót musí byť splnený aj minimálny počet účastníkov z miestnej populácie.

Grafické užívateľské rozhranie (GUI)

Po spustení programu sa užívateľ dostane na prihlasovaciu obrazovku, kde môže zadávať svoje prihlasovacie údaje. V prípade nesprávneho zadaného hesla alebo neexistujúceho používateľa sa zobrazí odpovedajúce chybové hlásenie.

Ak je prihlasovanie úspešné, užívateľ sa presunie na hlavnú obrazovku, ktorá obsahuje interaktívnu mapu Slovenska rozdelenú na kraje. Každý kraj je interaktívny prvok, na ktorý môže užívateľ kliknúť alebo kurzorom prejsť, aby získal bližšie informácie o danom kraji, ako je rozloha, počet obyvateľov či zaujímavé poznatky.

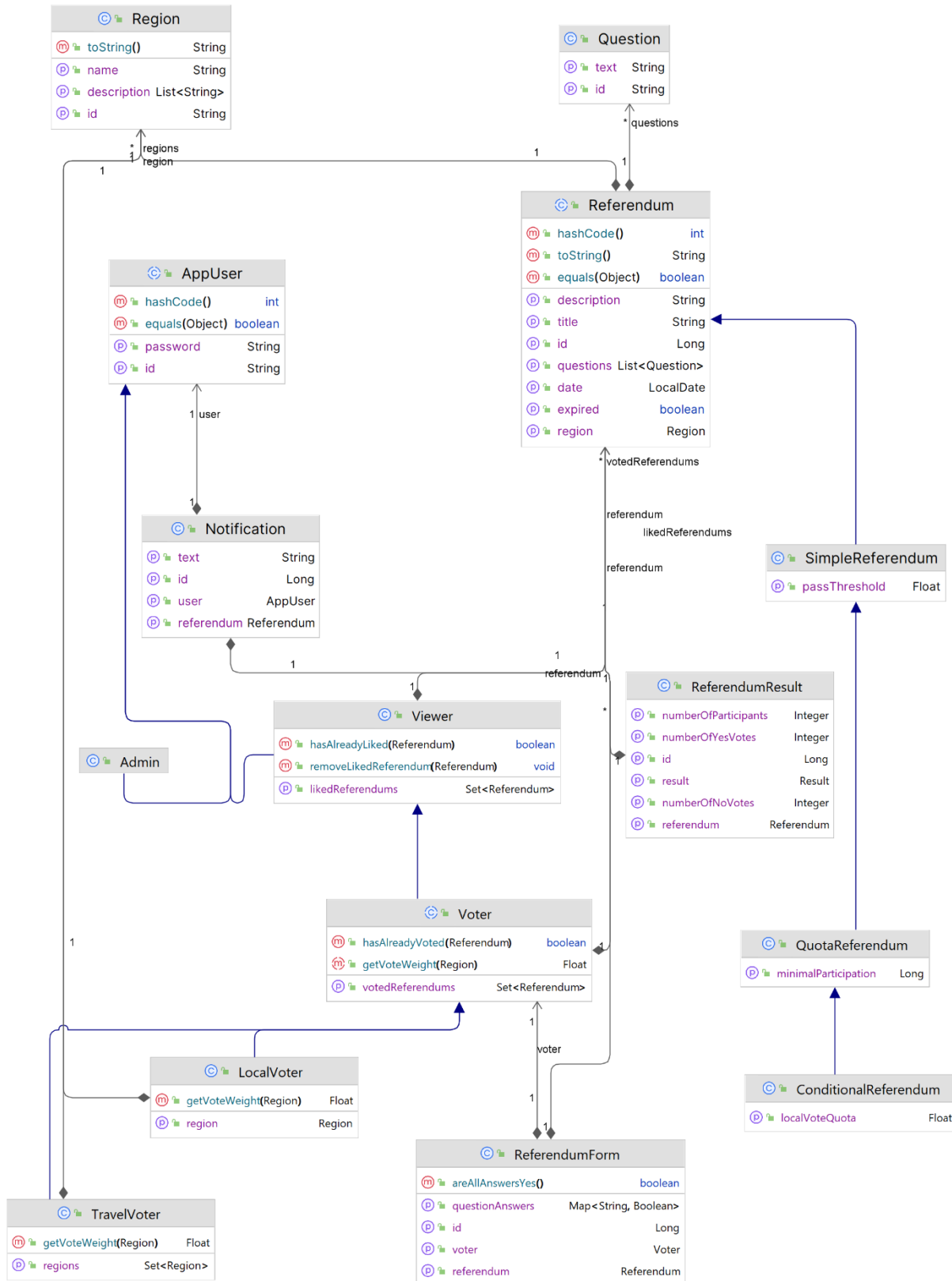
Okrem mapy sa na hlavnej obrazovke nachádza navigačná lišta, ktorá je dostupná na každej obrazovke v aplikácii. Táto lišta obsahuje tlačidlá, ktoré umožňujú užívateľovi odhlásiť sa, zobraziť výsledky referend a prezrieť si notifikácie. Súčasťou tejto navigačnej lišty je aj tlačidlo domov, ktoré užívateľa vráti na hlavnú obrazovku, a tlačidlo späť, ktoré mu umožní návrat na predchádzajúcu obrazovku.

Po výbere konkrétneho kraja sa užívateľ presunie na obrazovku s aktívnymi referendami pre daný kraj. Tu môže zobraziť zoznam referend, pričom každé referendum má možnosť ohodnotiť likom. Po ohodnotení sa užívateľovi zobrazí notifikácia, ak je referendum vyhodnotené. Taktiež ak má užívateľ právomoci, môže sa tiež zúčastniť daného referenda. No ak už sa zúčastnil predtým, zobrazí samu chybové hlásenie.

Pre administrátora sú k dispozícii ďalšie možnosti, ako je vyhodnotenie referenda alebo vytvorenie nového. V prípade vyhodnotenia referenda sa zobrazí správa všetkým používateľom, ktorí ohodnotili dane referendum. Pri vytváraní nového referenda administrátor vyplní potrebné informácie, ako je typ referenda, nadpis, popis a otázky, pričom musí zadať aj minimálny počet účastníkov a iné podmienky (závisí od typu referenda).

Celková navigácia a používateľské rozhranie aplikácie sú navrhnuté tak, aby poskytovali užívateľom jednoduchý a prehľadný spôsob interakcie s rôznymi funkciami a možnosťami aplikácie.

Class Diagramy a funkčný opis program



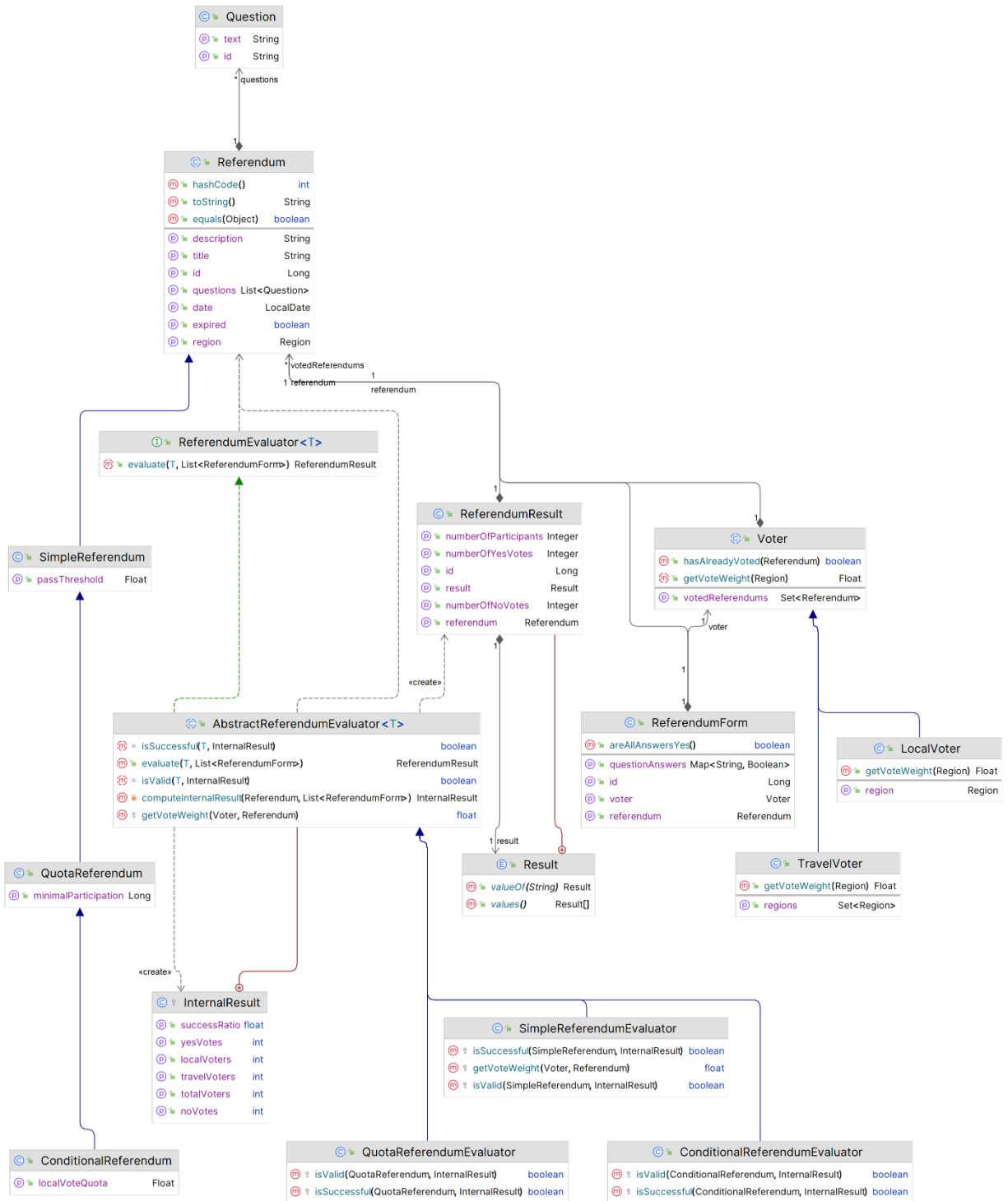
Táto aplikácia má jadro uložené v implementovanej databáze Spring, čo umožňuje ukladanie, vymazávanie a vyhľadávanie údajov uložených v nej. Databáza Spring poskytuje robustné nástroje na manipuláciu s dátami a efektívne riadenie ich ukladania a vyhľadávania. Týmto spôsobom je zabezpečená spoľahlivá a efektívna správa údajov pre našu aplikáciu.

Na prvom class diagrame je jasne viditeľné, že program je založený na dvoch hlavných hierarchiách: AppUser a Referendum. AppUser je základnou triedou, od ktorej sa dedí Admin a Viewer. Viewer je následne dedený na Votera, ktorý je rozdelený na LocalVoter a TravelVoter. Táto hierarchia úzko súvisí s regiónom, keďže sme potrebovali sledovať, z akého regiónu je každý Voter, aby sme mu mohli priradiť správnu váhu hlasu, čo zabezpečuje polymorfizmus, respkt. metóda getVoteWeight.

Dôležitou súčasťou je aj notifikácia, ktorá je prepojená s AppUser, čo nám umožňuje upozorniť používateľa, že referendum, ktorému dal like, bolo vyhodnotené.

Druhou hlavnou hierarchiou je Referendum, ktorá sa dedí na SimpleReferendum, z ktorého ďalej vychádza QuotaReferendum a ConditionalReferendum. Každý typ referenda má svoje jedinečné vlastnosti a preberá funkcie z predchádzajúceho typu. Napríklad SimpleReferendum obsahuje metódu getPassThreshold, ktorá určuje percentuálny počet hlasov za áno potrebný na platnosť referenda. QuotaReferendum, navyše s touto metódou obsahuje minimálny počet hlasujúcich, zatiaľčo ConditionalReferendum má aj minimálny počet miestnych Voterov, ktorí sa musia zúčastniť.

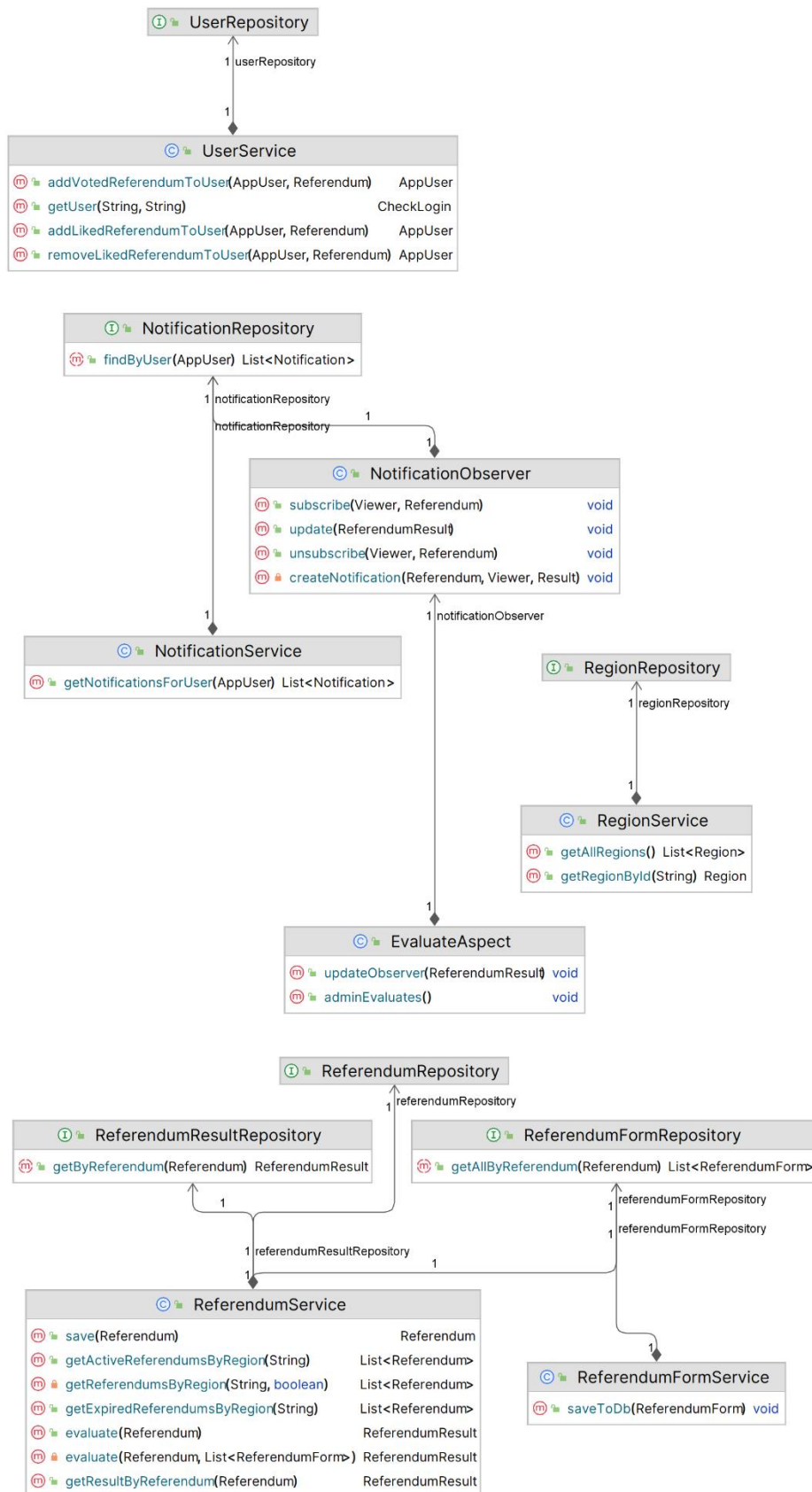
Referendum je tiež spojené s ReferendumForm, kde sa definujú otázky a detaily referenda, a s ReferendumResult, ktorý indikuje výsledky referenda, vrátane toho, či boli splnené požiadavky na jeho platnosť. Výsledky referenda sa získavajú z Evaluatorov, ktorých class diagram môžete vidieť tu:



ReferendumEvaluator je generická abstraktná trieda, ktorá zabezpečuje vyhodnotenie výsledkov referenda. Využíva návrhový vzor template method pattern, ktorý umožňuje flexibilné riešenie rôznych podmienok pre každý typ referenda. Každý typ referenda má svoju vlastnú podtriedu ReferendumEvaluator, ktorá obsahuje špecifické metódy na vyhodnotenie.

Tieto podtriedy obsahujú polymorfizmus metód isValid a isSuccessful na overenie platnosti a úspechu referenda podľa jeho typu. ReferendumEvaluator tiež obsahuje vnútornú triedu InternalResult, ktorá je zapuzdrená a obsahuje potrebné údaje na vyhodnotenie referenda. Táto trieda je chránená a poskytuje prístup iba zvnútra ReferendumEvaluator.

Ďalej sa pozrieme na repozitáre a servisy, ktoré sú prepojené na tomto class diagrame:



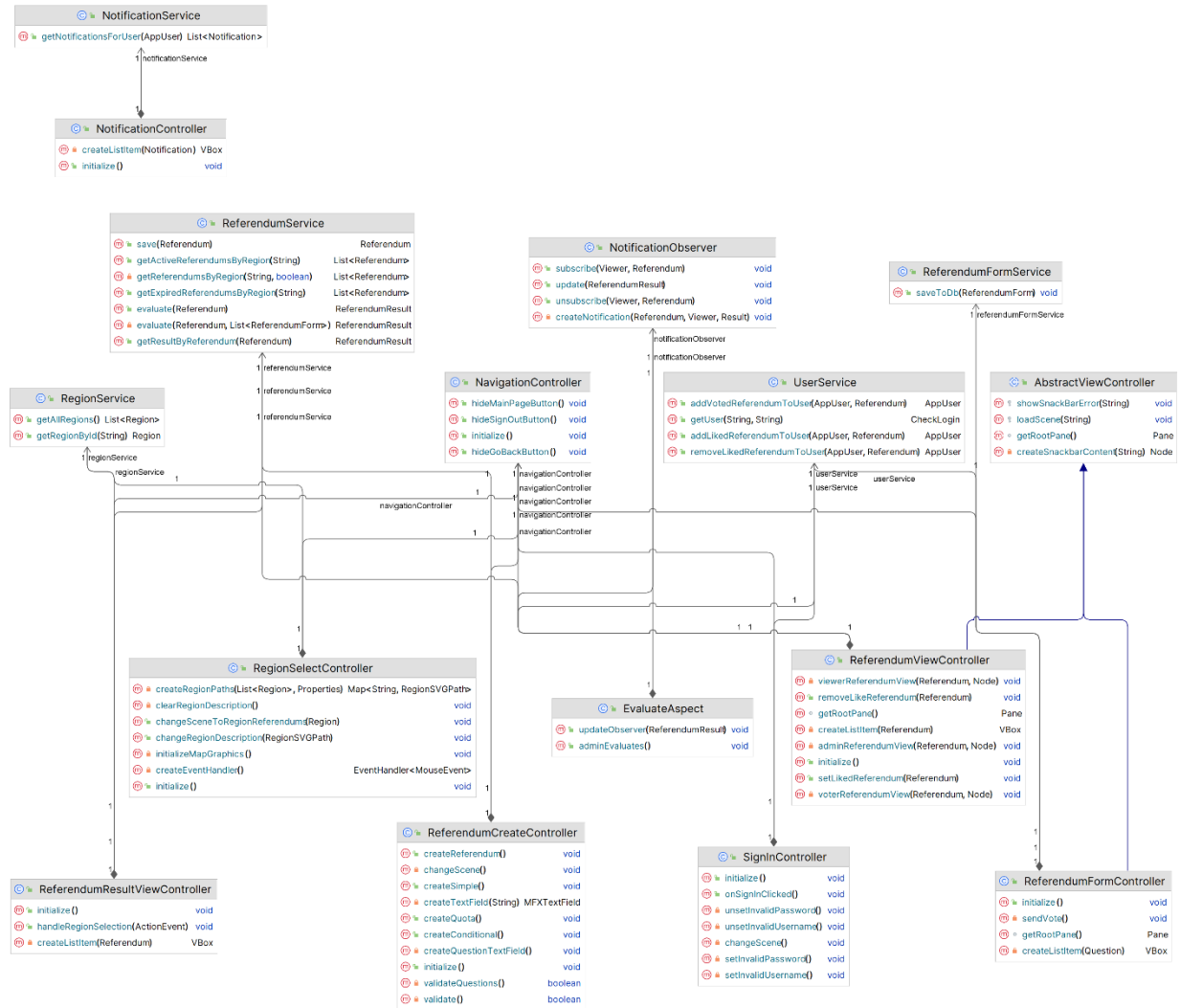
Hlavný repozitár tu máme pre UserService, kde spravujeme operácie ako pridávanie a odstraňovanie obľúbených referend pre používateľa, kontrola účasti na referende a overovanie prihlásenia používateľa. Ďalej tu máme repozitáre pre ReferendumService: ReferendumRepository, ReferendumResultRepository a ReferendumFormRepository.

Tieto repozitáre zabezpečujú manipuláciu s referendami, vrátane ukladania výsledkov v ReferendumResultRepository a vytvárania nových referend v ReferendumRepository. ReferendumFormRepository je zodpovedný za manipuláciu s otázkami a odpoveďami na dané otázky v rámci referenda. Tieto otázky a odpovede sú súčasťou vnútra referenda a ich správa je nevyhnutná pre jeho správne fungovanie.

Okrem toho existuje repozitár pre Region, ktorý slúži na získanie informácií o všetkých regiónoch alebo konkrétneho regiónu podľa identifikátora.

Ďalším dôležitým repozitárom je ten pre Notifikáciu, ktorý obsahuje implementáciu observera sledujúceho používateľov, ktorí si obľúbili určité referendum. Po vykonaní vyhodnotenia administrátorom sa použije aspekt na poslanie notifikácie všetkým používateľom, ktorí si dané referendum obľúbili. Každé referendum má vlastného observera, aby sme mohli správne posilať notifikácie iba tým používateľom, ktorí si ho obľúbili.

Ďalej sa pozrieme na prepojenie controllerov a servisov.



Controllery v našom projekte slúžia na riadenie užívateľského rozhrania (GUI). Pomocou FXML súborov definujeme, ako má vyzerať pre používateľa, pridávame tlačidlá, zoznamy a každému prvku priradíme vlastný identifikátor. V kontroleroch programujeme správanie týchto prvkov - čo majú robiť a čo má používateľ vidieť.

Pre každú scénu v programe máme vlastný kontroler. Tieto kontrolery potom získavajú údaje od našich servisov, ktoré tieto údaje poskytujú. Kontrolery majú svoje vlastné funkcie, ktoré sú zamerané na načítanie správnych údajov pre konkrétného používateľa.

Napríklad, kontrolér pre admina umožňuje vytvárať a vyhodnocovať referendá, čo je funkcionlita, ktorú iba admin môže využívať. Kontrolery zabezpečujú, že iba admin bude mať prístup k týmto špecifickým funkciám. Pre overenie, či je užívateľ admin, využívame službu pre používateľa, ktorá nám poskytne informácie, či máme zobrazíť príslušné funkcie alebo nie.

Splnenie kritérií:

V našom projekte sme úspešne splnili hlavné požiadavky. Máme dve dôležité hierarchie - jednu pre referendum a druhú pre používateľov aplikácie. Polymorfizmus je dôležitou súčasťou nášho projektu, čo vidíme napríklad v LocalVoter a TravelVoter, kde funkcia `getVoteWeight` priradzuje váhu hlasu v závislosti od konkrétneho referenda. Ďalšie príklady polymorfizmu vidíme pri vyhodnocovaní referend, kde sa využívajú funkcie `isValid` a `isSuccessful` v `AbstractReferendumEvaluator`, `SimpleReferendumEvaluator`, `QuotaReferendumEvaluator` a `ConditionalReferendumEvaluator`.

Z hľadiska zapúzdrenia sme sa držali v `AbstractReferendumEvaluator`, kde sme údaje označili ako `protected` a nazvali sme ich `InternalResult`, čím sme zabezpečili ich bezpečný prístup. Agregácia je ďalším dôležitým kritériom, ktoré sme dodržali, ako napríklad v triede `Referendum`, kde sme mali agregáciu k objektu `Region`. Aj u `TravelVoter` sme mali ďalšiu agregáciu, kde sme tiež priradili región.

Naša aplikácia úspešne splnila ďalšie kritériá vrátane vlastných návrhových vzorov. `Template Method Pattern`, implementovaný v `AbstractReferendumEvaluator`, umožňuje flexibilné riešenie rôznych podmienok pre každý typ referenda. `Observer`, použitý v `NotificationObserver`, sleduje používateľov, ktorí prejavili záujem o určité referendum, a je úzko prepojený s ďalším splneným kritériom, a to `Aspectom`, ktorý slúži na odoslanie notifikácií používateľom po vyhodnotení referenda administrátorom. Grafické rozhranie je ďalšou dôležitou súčasťou nášho projektu.

Ďalším splneným kritériom je RTTI (`Run-Time Type Information`), ktoré využívame v `ReferendumService` na identifikáciu typu referenda pomocou `instanceof`. Prítomnosť vnútornej triedy `InternalResult` v `AbstractReferendumEvaluator` prispieva k ďalšiemu splnenému kritériu (`Vhniezdenej triedy`). Táto trieda je taktiež generická, čo tiež spĺňa ďalšie kritérium.

Naša aplikácia obsahuje dve vlastné výnimky. Prvou je `IOException`, ktorá môže nastať pri načítaní FXML súboru. Túto výnimku chytáme a vyhadzujeme vlastnú výnimku, ktorú ošetrujeme pomocou `snackbaru`, aby sme upozornili používateľa na chybu. Tento mechanizmus je implementovaný v celej aplikácii.

Druhou výnimkou je `UserException` v `UserService`, ktorá sa vyskytuje napríklad pri pokuse o hlasovanie používateľa. Kontroluje sa, či je používateľ oprávnený voliť, a ak už volil, vyhodí sa výnimka `UserException`. Táto výnimka je taktiež spracovaná pomocou `snackbaru`, čo umožňuje ošetrovanie výnimky priamo v užívateľskom rozhraní. `UserException` je vlastná výnimka, ktorú vyhadzujeme v prípade, že aplikačná logika nesedí - napríklad, keď používateľ, ktorý nie je oprávnený voliť, sa pokúsi voliť.